

B8 Series B

Clustering: resolution sweeps and clustree

The number of clusters is not something the data tells you — it is something you ask. Learn to ask well.

About 38 min | Prerequisites: B7 | Data: PBMC 3k

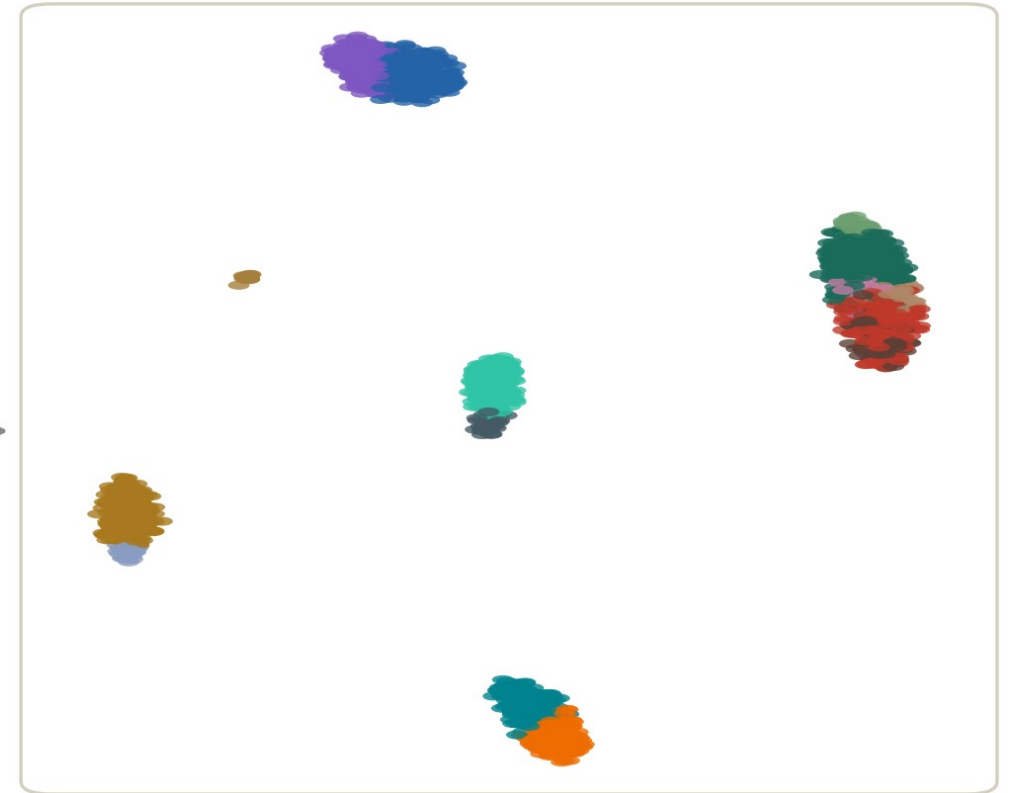
Same data, two resolutions

low resolution → 9 clusters



simulated data

high resolution → 15 clusters



same
dataset



both run fine and both look plausible — which do you pick, and why?

Resolution 0.5 gives 9 clusters; 1.2 gives 15. Which one is 'real'?

WHERE THIS EPISODE STARTS

'Why this resolution?'

'It is the Seurat default' is not an answer.

Reviewers really do ask. This episode gives you a procedure that produces an answer.

What you will learn

- 1 Explain what FindNeighbors and FindClusters each do, and what k.param, dims and resolution mean
- 2 Sweep resolution 0.2–1.4 and read stable vs turbulent layers on a clustree
- 3 Validate each extra cut with markers and per-cluster QC, then finalize and record the reason

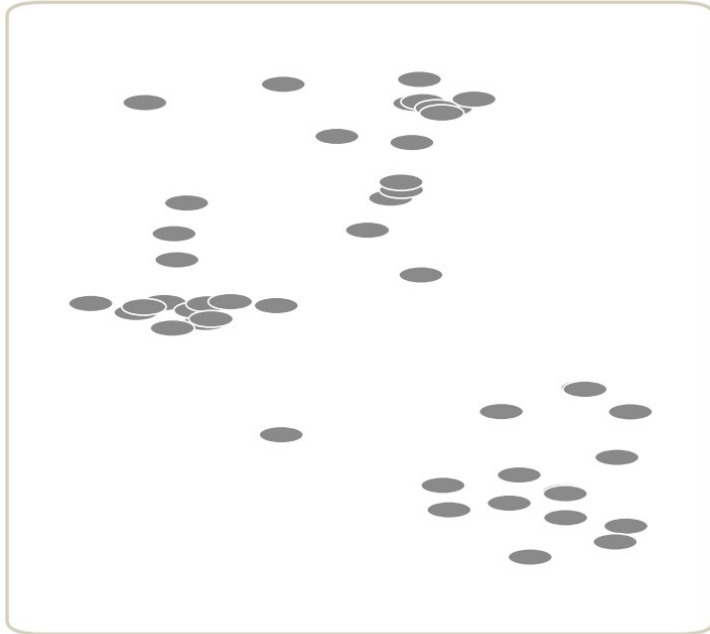
01

What clustering actually does

It proposes hypotheses — it does not discover truth

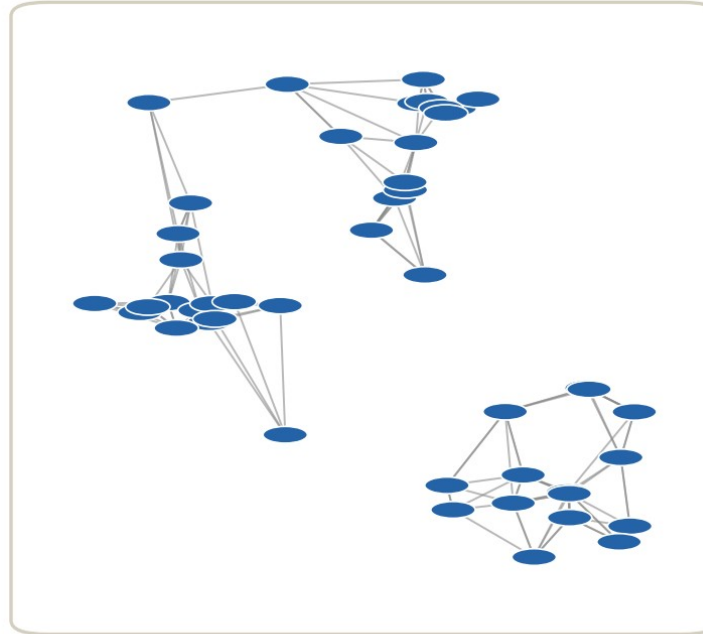
From points to a graph to communities

1. points: cells in PC space



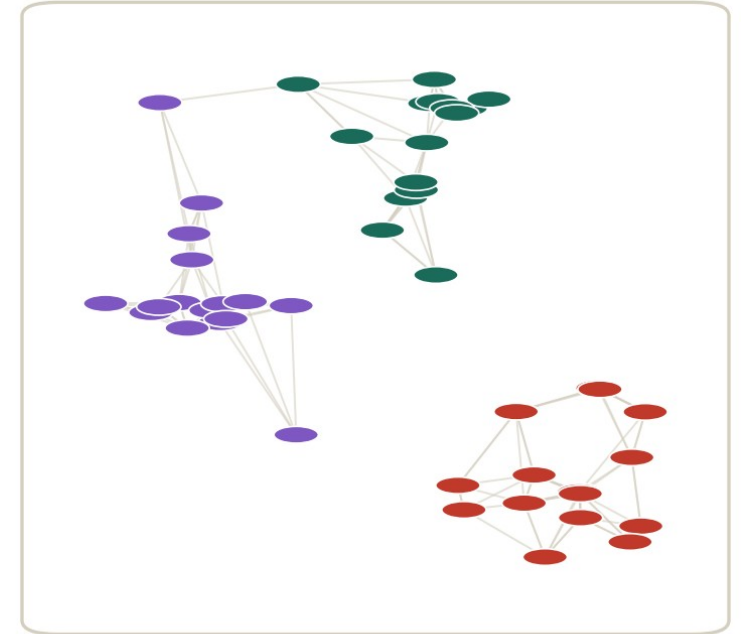
coordinates, no structure yet

2. graph: k nearest neighbours



FindNeighbors (k.param)

3. communities: dense inside



FindClusters (resolution)

clustering never touches the embedding — it happens on the KNN graph

FindNeighbors builds the graph; FindClusters finds communities that are dense inside, sparse outside

Resolution: the magnification dial



all three are the same data — no power is 'correct', only fit for your question

- low power: only continents — T, B, monocytes
- high power: continents split into subsets
- higher still: you start cutting noise
- there is no correct power — only one that fits your question

CORE IDEA

**'The correct number of clusters' does not exist.
What exists is structure that is stable at a given
resolution.**

Clustering outputs a list of hypotheses awaiting marker and QC evidence — not answers.

02

Two lines, two steps

FindNeighbors builds the graph → FindClusters cuts communities

FindNeighbors: wire the cells into a graph

```
pbm c <- readRDS("output/pbm c_b05_qc.rds") # 承接 B5-B7 的物件  
pbmc <- FindNeighbors(pbm c, dims = 1:10) # dims: B7 選的 nPC
```

Computing nearest neighbor graph
Computing SNN

k.param defaults to 20 nearest neighbours, measured in PC space — not gene space

Three things to know about FindNeighbors

dims = 1:10



inherited from B7's
nPC choice — change
it and everything
downstream changes

k.param = 20 (default)



smaller fragments
clusters, larger
smooths them away;
leave it unless you
have a reason

Output: two graphs



RNA_nn and RNA_snn
land in the object;
clustering uses the
SNN

FindClusters: cut the graph into communities

```
pbm c <- FindClusters(pbm c, resolution = 0.5)
```

Modularity Optimizer version 1.3.0 by Ludo Waltman and
Nees Jan van Eck

Number of nodes: 2638
Number of edges: 95927

Running Louvain algorithm ...
Maximum modularity in 10 random starts: 0.8723
Number of communities: 9

Elapsed time: 0 seconds

algorithm = 1 (Louvain) is the default; resolution is this episode's protagonist

The algorithm argument: Louvain vs Leiden

Both are community detection; the maths lives in Series C

Louvain

algorithm = 1, the Seurat default

- fast, stable, battle-tested for a decade
- known flaw: can output internally disconnected communities
- what we use throughout this episode

Leiden

algorithm = 4, the refinement

- guarantees well-connected communities (Traag 2019)
- the default in the scanpy world
- in Seurat it needs the Python leidenalg installed

Inspect: a new column, a new identity

```
table(Ids(pbm c))
```

```
DimPlot(pbm c, reduction = "um ap", label= TRUE)  
# UMAP 是 B9 的主角，今天先借它的座標看結果
```

0	1	2	3	4	5	6	7	8
684	481	476	344	291	162	155	32	13

Numbering starts at 0, ordered by size. Labels are arbitrary — the traps section will collect on this

What this section did

One SNN graph



FindNeighbors wired
2,638 cells in 10-PC
space

Nine communities



FindClusters cut 9
clusters at resolution
0.5, stored in
seurat_clusters

Nine hypotheses



none verified yet; 0.5
is not final — next we
start interrogating

03

The resolution sweep and clustree

Do not pick one value — look at the whole spectrum first

Sweep 0.2 to 1.4 in one loop

```
for (r in seq(0.2, 1.4, by = 0.2)) {  
  pbmc <- FindClusters(pbmc, resolution = r, verbose = FALSE)  
}  
  
grep("^RNA_snn_res", colnames(pbmc@meta.data), value = TRUE)
```

```
[1] "RNA_snn_res.0.5" "RNA_snn_res.0.2" "RNA_snn_res.0.4"  
[4] "RNA_snn_res.0.6" "RNA_snn_res.0.8" "RNA_snn_res.1"  
[7] "RNA_snn_res.1.2" "RNA_snn_res.1.4"
```

One metadata column per resolution, all kept. Note 1.0 becomes 'res.1' in the name

How many clusters at each resolution

```
res.cols <- grep("^RNA_snn_res", colnames(pbm.c@meta.data),  
                value = TRUE)  
sapply(pbm.c@meta.data[res.cols], \(x) length(unique(x)))
```

RNA_snn_res.0.5	RNA_snn_res.0.2	RNA_snn_res.0.4	RNA_snn_res.0.6
9	6	8	10
RNA_snn_res.0.8	RNA_snn_res.1	RNA_snn_res.1.2	RNA_snn_res.1.4
11	13	15	16

Anything from 6 to 16 clusters. Also: the active identity is now stuck at the last run, 1.4

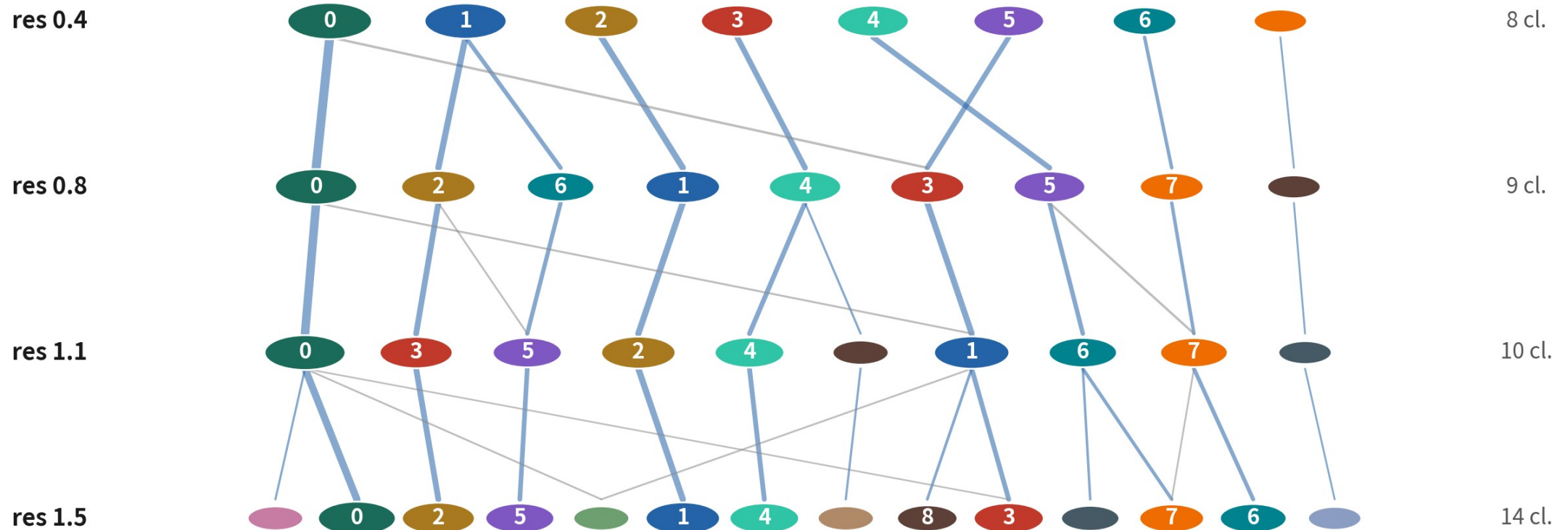
clustree: draw the sweep as a tree

```
# install.packages("clustree")  
library(clustree)  
  
clustree(pbm c, prefix = "RNA_snn_res.")
```

Point prefix at the shared column-name stem; each resolution becomes a layer, cells flow between them

This is a clustree

one row per resolution; node size = cells; edge width = cells flowing



simulated data (KNN graph + Louvain, actually run)

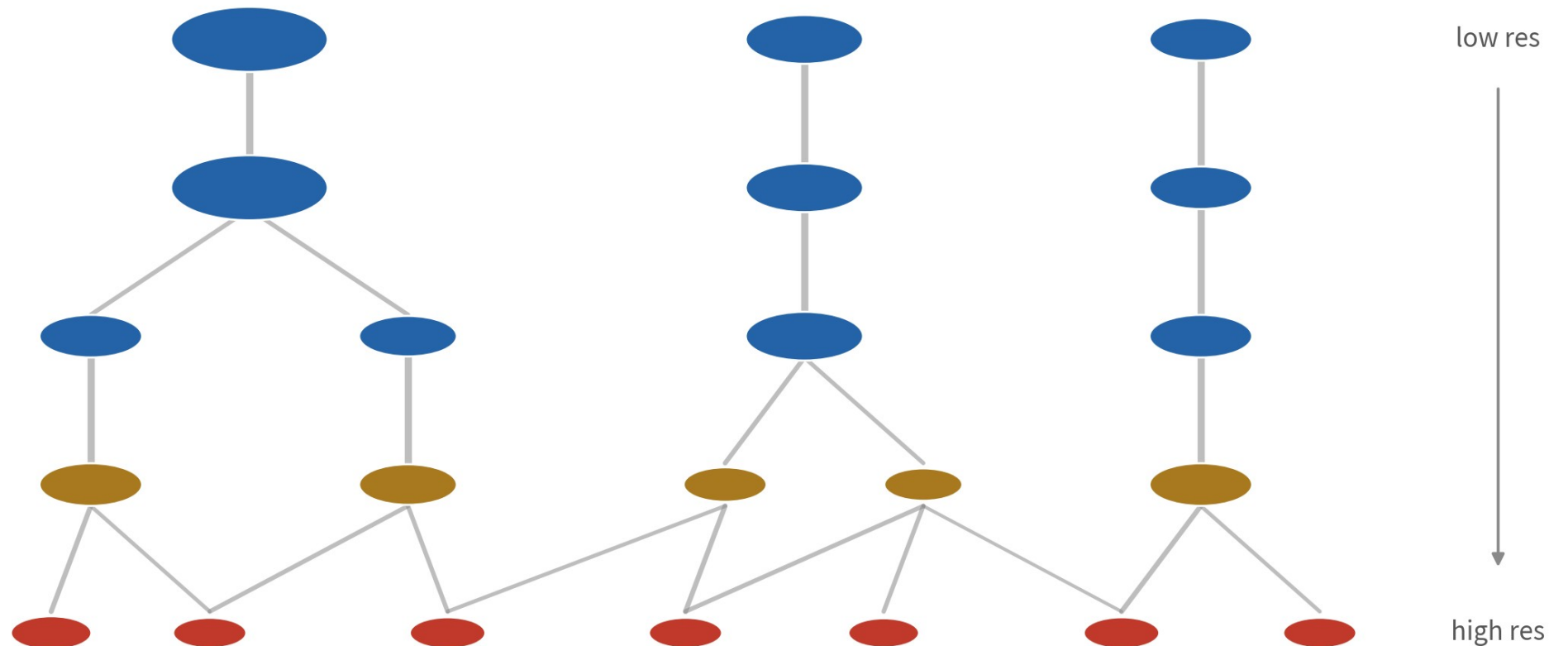
One row per resolution; nodes are clusters sized by cell count; arrows show where cells flow

How to read it: stable vs turbulent layers

stable:
unchanged
across res

clean split:
go verify

turbulent:
crossing, swaps
= too far



finalize at the deepest layer that is still stable — below it, turbulence

The upper layers are stable with clean splits; further down arrows cross and cells swap — you have cut too far

Three questions as you read down the tree

Is this layer stable?



layers unchanged
across consecutive
resolutions are
candidates

Is the split one-way?



one node splitting
cleanly into two, no
backflow — worth a
marker check

Is there swapping?



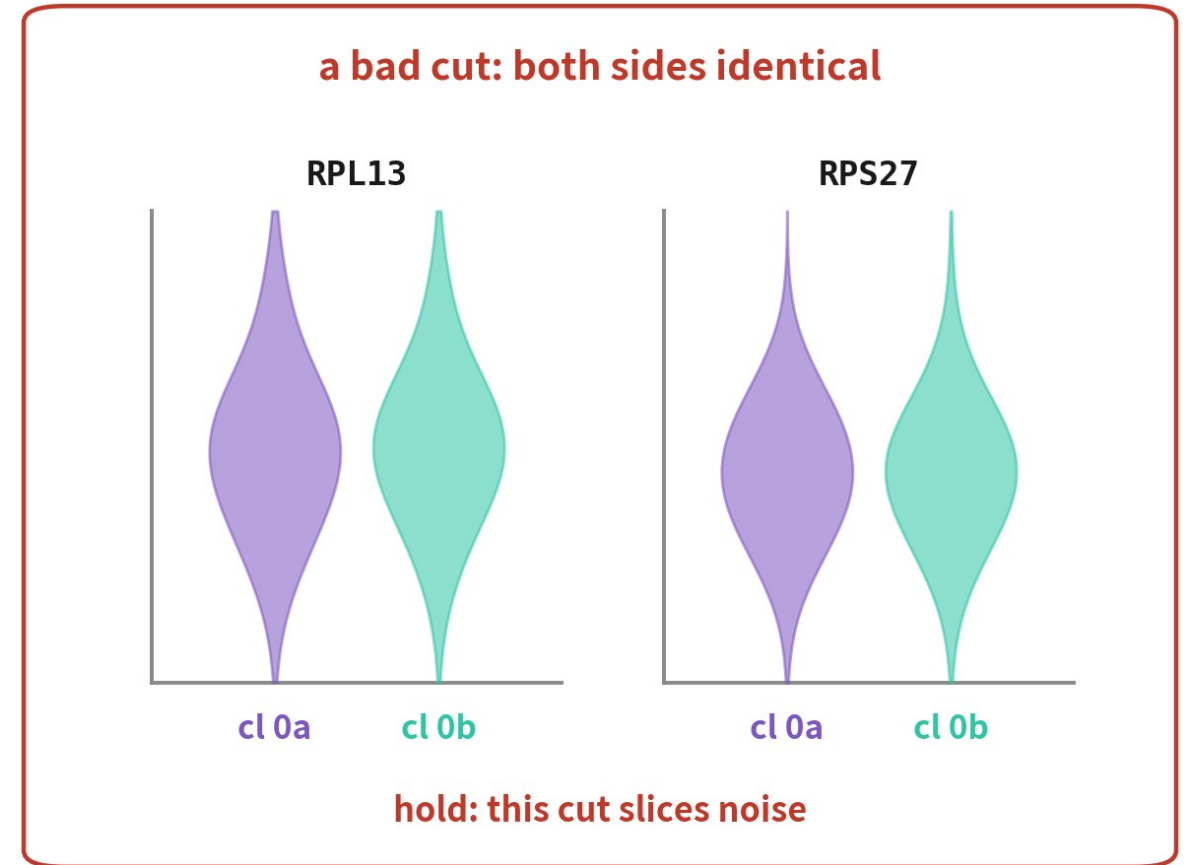
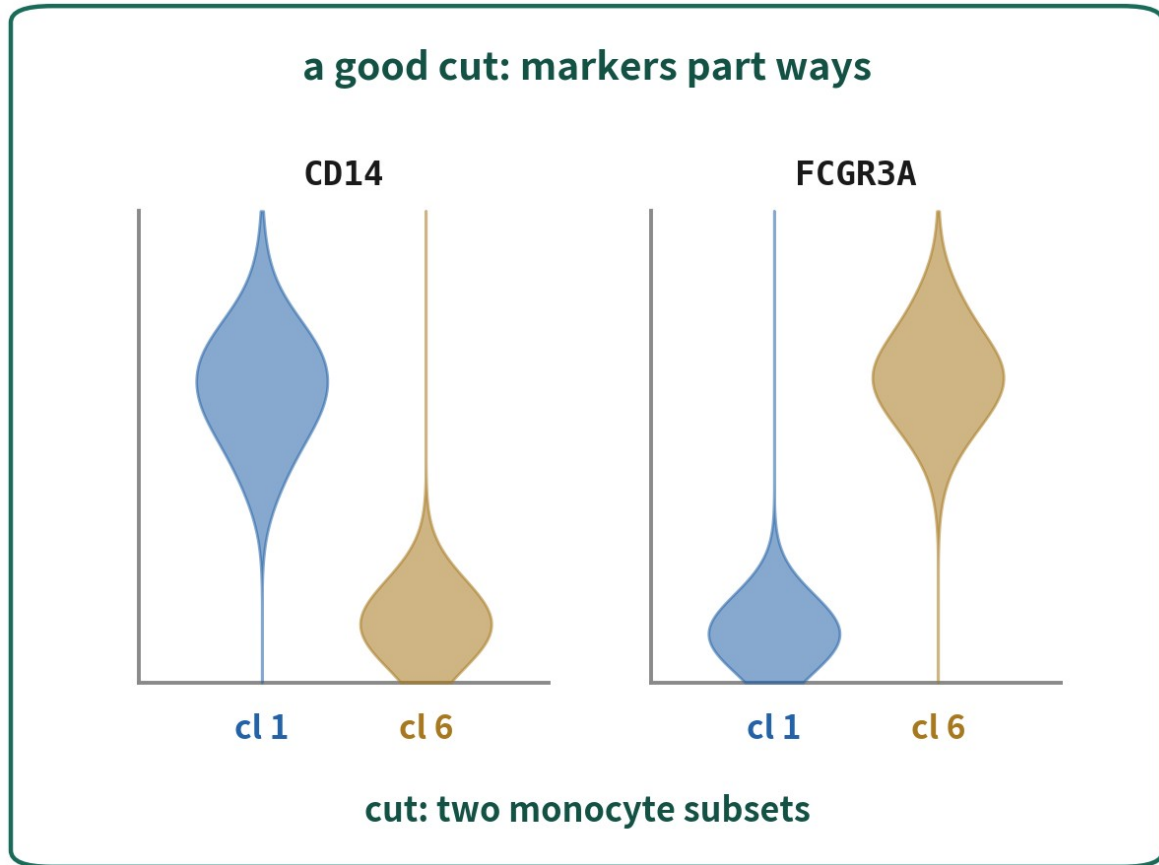
crossing arrows,
clusters fed by many
parents — turbulence,
back off

04

Validate the cut with markers

clustree says 'stable'; only biology can say 'meaningful'

The extra cut between adjacent resolutions



simulated data

Both are 'one cluster becomes two': left, two marker sets part ways; right, the two profiles are identical

First, find which cluster got cut

```
# 相鄰兩個 resolution 的交叉表：一列一個舊群  
table(pbm c$RNA_snn_res.0.4, pbm c$RNA_snn_res.0.6)
```

	0	1	2	3	4	5	6	7	8	9
0	644	0	0	0	0	0	0	40	0	0
1	0	436	0	0	0	0	158	0	0	0
2	0	0	476	0	0	0	0	0	0	0
...										

Cluster 1 at 0.4 splits into 436 + 158 cells at 0.6 — that is the cut we interrogate

What separates the two halves

```
idents(pbm c) <- "RNA_snn_res.0.6"  
split.m k <- FindMarkers(pbm c, ident.1 = 1, ident.2 = 6,  
                        m.in.pct = 0.25)  
head(split.m k, 4)
```

	p_valavg	log2FC	pct.1	pct.2	p_val_adj
S100A8	3.1e-172	3.90	0.975	0.229	4.3e-168
S100A9	8.4e-165	3.51	0.996	0.400	1.2e-160
LGALS2	6.0e-151	2.87	0.908	0.078	8.2e-147
CD14	2.2e-106	1.82	0.667	0.028	3.0e-102

CD14 spikes on one side; the reverse list holds FCGR3A and MS4A7 — textbook monocyte subsets

Cross-examine on the plots

```
FeaturePlot(pbm c, features = c("CD 14", "FCGR3A"))
```

```
VlnPlot(pbm c, features = c("CD 14", "LYZ", "FCGR3A", "MS4A7"),  
        idents = c(1, 6), ncol = 4)
```

FeaturePlot checks the cut's location; VlnPlot checks the two clusters truly part ways

To cut or not to cut

The same checks on two cuts give two different verdicts

the 0.4 → 0.6 cut

Marker-backed: cut

- CD14/LYZ vs FCGR3A/MS4A7 — two marker sets, two clusters
- matches known biology: classical vs non-classical monocytes
- this cut makes the data say one more true thing

the 0.6 → 0.8 cut

No evidence: hold

- top genes all ribosomal, log2FC under 0.5
- pct.1 and pct.2 both 99% — everyone expresses them
- the extra cut just halves the noise

05

Before finalizing: the QC autopsy

Collecting the two hooks B5 left behind

Per-cluster QC: junk clusters step forward

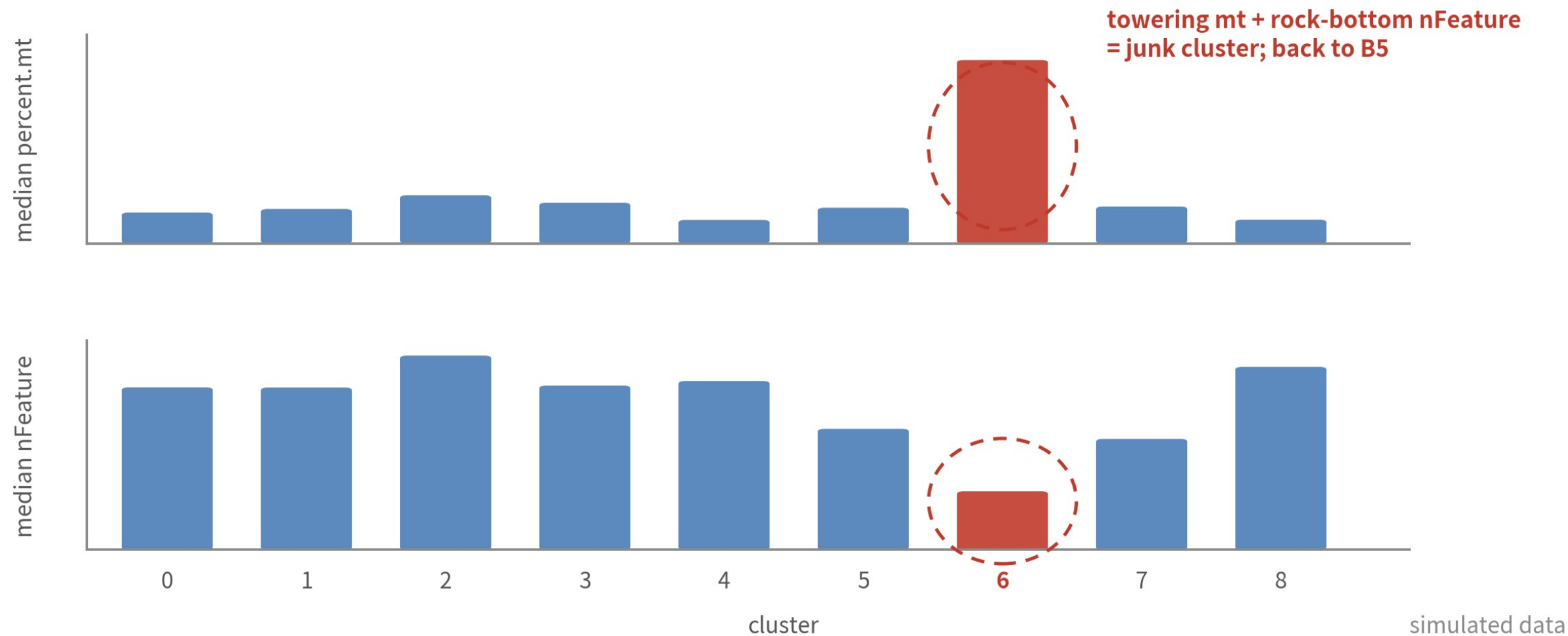
```
library(dplyr)

qc.tab <- pbmc@meta.data %>%
  group_by(cluster = RNA_snn_res.0.6) %>%
  summarise(n = n(),
            mt = median(percent_mt),
            nF = median(nFeature_RNA))
arrange(qc.tab, desc(mt)) %>% head(3)
```

```
# A tibble: 3 x 4
  cluster    n    mt   nF
  <fct>    <int> <dbl> <dbl>
1 8         15  3.4  692
2 4        291  3.1  814
3 5        206  2.9  889
```

You are looking for a cluster with towering mt and rock-bottom nFeature. Here: none

What a junk cluster looks like



Median mt 12%, half everyone else's nFeature, stress-gene markers — that is not a novel cell type

Per-cluster doublet share: cashing in the B5 labels

```
df.col <- grep("^DF.classifications",  
              colnames(pbm.c@meta.data), value = TRUE)  
  
round(prop.table(table(pbm.c$RNA_snn_res.0.6,  
                      pbm.c@meta.data[[df.col]]), 1), 3)[1:5, ]
```

	Doublet	Single
0	0.011	0.989
1	0.018	0.982
2	0.008	0.992
3	0.015	0.985
4	0.021	0.979

All hug the global 2.3% expectation — pass. What you are hunting is a cluster where half are flagged

What a doublet cluster looks like



38% flagged and wedged between two big clusters — the cluster does not contain doublets, it is doublets

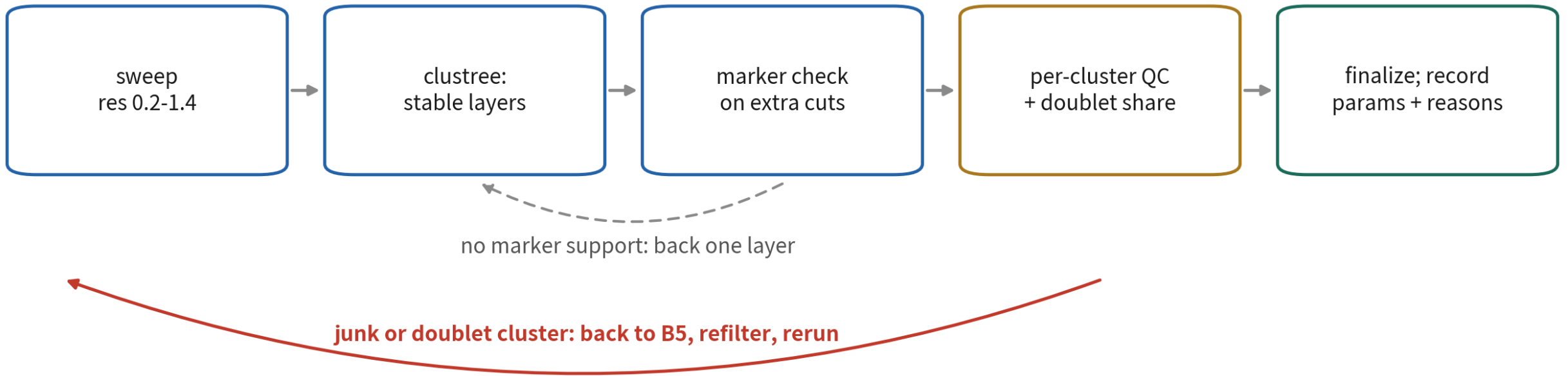
FOR YOUR METHODS SECTION

**"FindNeighbors (dims 1:10) + Louvain; resolutions
0.2-1.4 swept; 0.6 chosen for clustree stability
and marker support."**

Parameters, sweep range, and the reason — all three. The word 'default' leaves your methods section for good.

The whole decision loop, one picture

you do not pick a resolution — it is what survives screening and verification



Sweep → clustree → marker check → per-cluster QC → record; junk clusters send you back to B5

06

Where people get hurt

Three traps — each demonstrated for real

Trap 1: dial until you get the number you wanted

What it is

'This tissue should have 12 cell types', so you tune resolution until you get exactly 12 clusters.



Why it happens

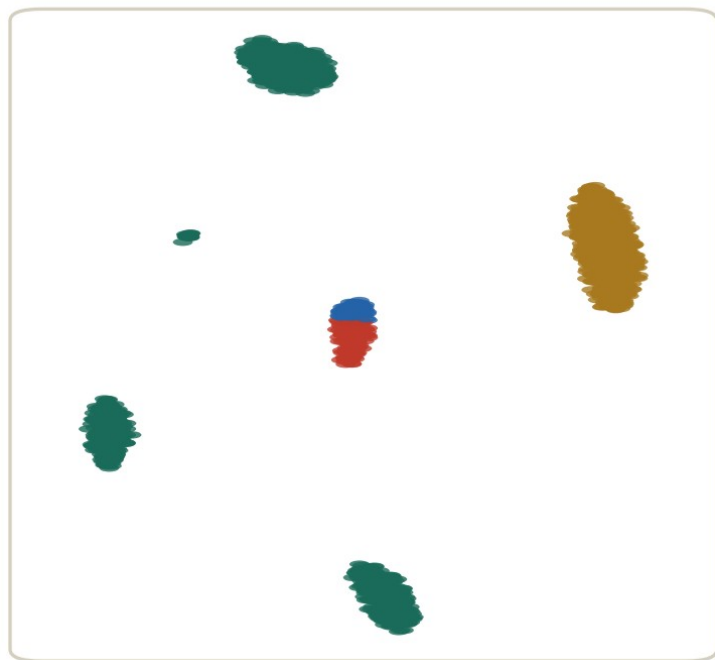
It feels like validation — the count matches the literature.

What it costs you

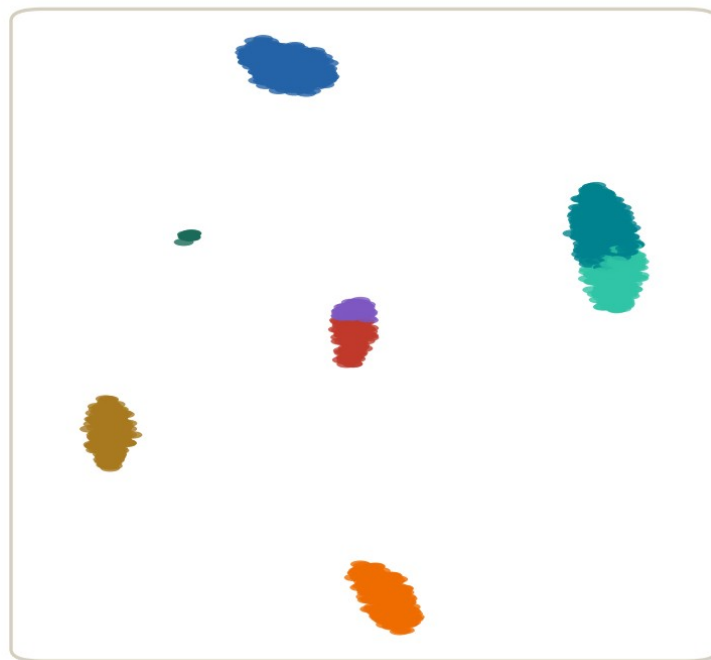
Circular reasoning: you picked the parameter with the conclusion, then used it to produce the conclusion. The sweep already showed 6 to 16 clusters on demand.

Any number of clusters, made to order

you asked for 4



you asked for 8



you asked for 16



simulated data

the same data 'successfully' yields 4, 8 or 16 clusters — the count is never evidence

Same data: ask for 4, 8 or 16 clusters and the algorithm obliges. The count itself is not evidence

Trap 2: comparing cluster IDs across runs

What it is

Rerun with different dims or k.param, then claim 'cluster 3 is still cluster 3, so results are stable'.



Why it happens

Same number, so intuitively the same cluster.

What it costs you

IDs only reflect size ordering; 'cluster 3' can be entirely different cells. Cross-tabulate runs — never match by ID.

Two 'cluster 3's, two different populations

run A (k.param=15): cluster 3



run B (k.param=8): cluster 3



same ID,
different cells

match with
table(A, B)

simulated data

IDs are just a size ranking — change a parameter and the ranking reshuffles everything

The one-line check: `table(old, new)`. A scrambled diagonal means the labels have been reshuffled

Trap 3: publishing a junk cluster as a novel cell type

What it is

An extra small cluster with 'distinctive' markers goes straight into the paper as a discovery.



Why it happens

It genuinely forms its own cluster, and FindAllMarkers will happily hand it a marker list.

What it costs you

High mt, low nFeature, markers all mitochondrial and heat-shock genes — dying cells clustering together. The fix is not deleting it; it is going back to B5 and refiltering.

TRAPS, SUMMED UP

**Counts can be ordered, IDs get reshuffled,
and junk clusters too.**

Your only defence is evidence, not parameters.

Marker evidence, QC metrics, doublet share — a cluster only counts after all three checks.

5+

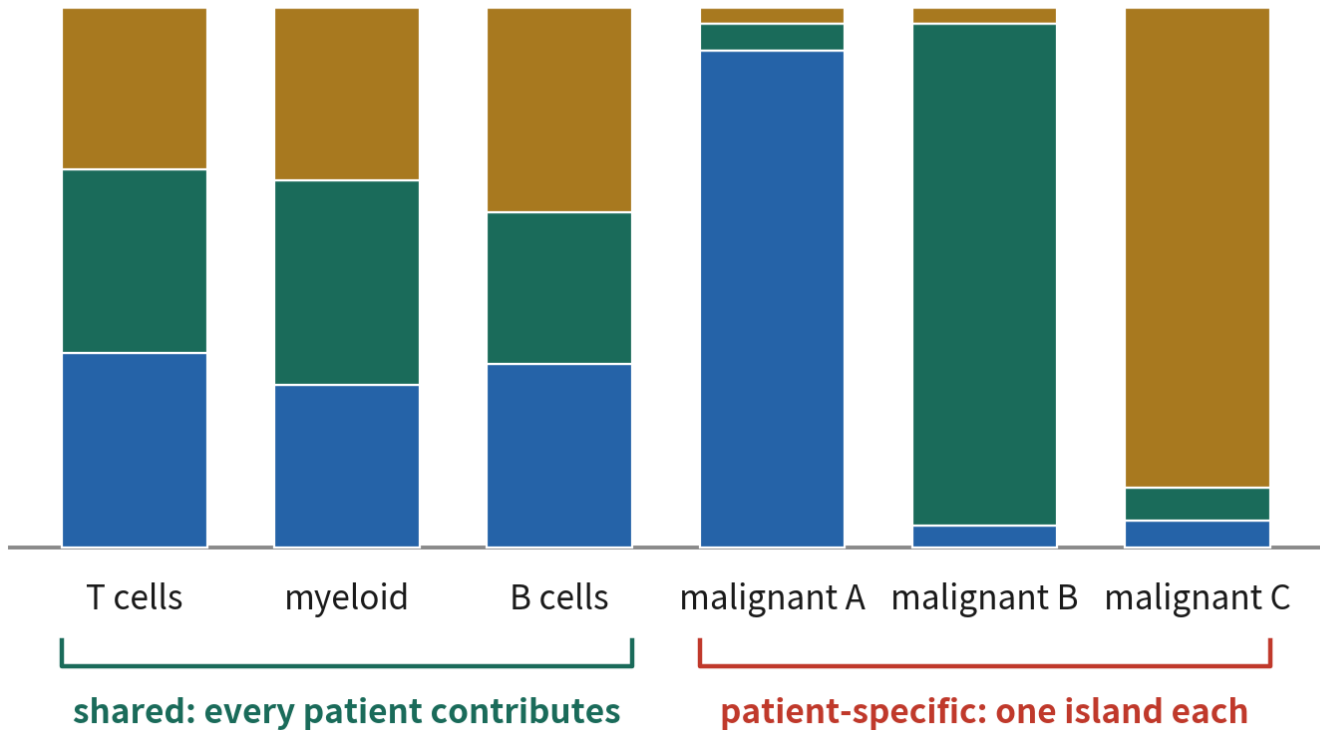
Strategy Room: Complex Samples

The standard route is done — now imagine you hold not PBMC, but tumours from five patients

The tumour-standard column: patient composition

● patient 1 ● patient 2 ● patient 3

patient composition per cluster (fraction)



schematic

the code

```
table(cluster, patient)  
— slide 25's cross-tab, second axis = sample
```

the reading

malignant islands per patient are normal:
CNVs differ per patient — not a batch failure

the decision

add a patient-composition column to the
per-cluster QC table — B10/B11/B12 all read it

TME clusters draw from every patient; malignant clusters are one island each. `table(cluster, patient)` shows it — normal, not a batch failure

Coarse first, fine later: hierarchical clustering

Layer 1: coarse cut for lineages



low resolution
separates immune /
stroma / malignant;
the top clustree layers
are usually stable —
finalize and annotate
them first

Layer 2: subset and rerun



for T-cell subsets:
subset, then rerun
HVG, PCA and
clustering — not a
higher resolution on
the full object

Two brakes



do not subcluster
below ~200 cells;
reusing the full
dataset's PCs after
subsetting is a classic
mistake

Inside the malignant continent: what is that cut?

Clone, state or noise — on a clustree they wear the same face

three possibilities

resolution cannot tell them apart

- clone: subclones with different CNVs — genuine genomic difference
- state: cycling, hypoxia, stress — moods of the same cells
- noise: the marker table blends state genes with CNV effects

the strategy

harder evidence, not higher resolution

- clones: wait for the CNV evidence in B16
- states: score published gene sets — continuous, comparable across patients
- today's label stops at 'malignant + patient'

Necrotic samples: junk or biology? Three lines of evidence

1. QC, relative



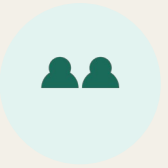
compare against same-
lineage neighbours,
not the whole dataset
— malignant mt
baselines run high

2. Marker content



strip mito and heat-
shock genes: lineage
markers and a
coherent hypoxia
signature left? junk
keeps nothing

3. Recurrence across samples



a biological cluster
recurs across patients
and necrotic-core
samples; junk tracks
one sample

Episode checklist



dims inherited from B7; k.param default on purpose



resolution 0.2-1.4 swept; all results kept in metadata



clustree drawn; stable and turbulent layers located



every cut you kept has marker evidence behind it



per-cluster QC metrics and doublet shares inspected



resolution, parameters and reasons in your notes

All six boxes ticked before you move to B9

ONE-LINE SUMMARY

**Clustering gives hypotheses.
clustree finds the stable layers, markers decide,
and you write down why.**

Next time a reviewer asks 'why 0.6', you answer with a sweep, a tree and a marker table.

Check yourself 1

What do FindNeighbors and FindClusters each do?

- A PCA, then UMAP
- B Build the KNN/SNN graph in PC space; run community detection on it
- C Filter cells; set the cluster count
- D Both compute cell distances with different algorithms

Answer B. FindNeighbors wires each cell to its PC-space neighbours; FindClusters runs Louvain or Leiden to find dense communities. The cluster count is an output, never an input.

Check yourself 2

On a clustree, which pattern says the resolution has probably gone too far?

- A Node counts grow with resolution
- B A node splits cleanly into two
- C Crossing arrows — cells swapping between clusters across layers
- D More clusters at the bottom than the top

Answer C. More clusters is the point of sweeping, and clean one-way splits deserve marker checks; but cells swapping back and forth means the boundaries are no longer stable — turbulence.

Check yourself 3

The 0.8-vs-0.6 extra cut yields only ribosomal top genes, log2FC under 0.5, pct 99% on both sides. Your call?

- A Take 0.8 — more clusters, more information
- B Take 0.6 — the extra cut has no biological support
- C Split the difference and rerun at 0.7
- D Switch to Leiden first

Answer B. Ribosomal genes are expressed by everyone; pct 99% on both sides means the cut slices a continuum, not a type boundary. No marker, no cut — stay at 0.6 and record why.

NEXT EPISODE

B9: UMAP / t-SNE — parameters, randomness, reproducibility

Clusters are final, but on paper they are still just numbers. Next we draw them in 2D — run the same line twice: do you get the same UMAP? And if not, what about your figures?